

Mathematics of Computing

Sayona C Sajan

CUSAT – M.Tech Artificial Intelligence & Software Engineering

2026

1. Proofs – Questions

- ① **Computing:** A recursive program has $T(n) = T(n - 1) + 1$, $T(1) = 1$. Prove that $T(n) = n$.
- ② Prove that the sum of two even integers is even.
- ③ Why can testing a finite number of cases never prove a universal mathematical statement?
- ④ What are the essential components of a mathematical proof?
- ⑤ Why is the argument “all tested primes greater than 2 are odd, therefore every prime is odd” invalid?

1. Proofs – Answers

- ① **Induction:** Base case $T(1) = 1$. Assume $T(k) = k$. Then

$$T(k + 1) = T(k) + 1 = k + 1.$$

Therefore $T(n) = n$.

- ② Let $a = 2m$ and $b = 2n$. Then

$$a + b = 2(m + n),$$

which is even.

- ③ Finite testing only establishes that the statement is true for the tested cases. A universal statement requires a proof for every allowed case.
- ④ Assumptions, definitions, logical reasoning, justified intermediate steps, and a conclusion.
- ⑤ The number 2 is a counterexample. A universal statement fails if even one valid counterexample exists.

2. Propositions – Questions

- 1 **Computing:** A system accepts a request only when the input is valid and the user is authenticated. Express this as a proposition.
- 2 What is a proposition?
- 3 When is an implication $p \rightarrow q$ false?
- 4 Express “If the server is down, the application cannot connect” symbolically.
- 5 Rewrite $p \rightarrow q$ using only $\neg$ and $\vee$.

2. Propositions – Answers

- ① If $p = \text{valid input}$ and $q = \text{authenticated user}$, then

$$p \wedge q.$$

- ② A proposition is a declarative statement that is either true or false, but not both.

- ③ $p \rightarrow q$ is false only when

$$p = T, \quad q = F.$$

- ④ If $p = \text{server is down}$ and $q = \text{application can connect}$:

$$p \rightarrow \neg q.$$

- ⑤

$$p \rightarrow q \equiv \neg p \vee q.$$

3. Predicates and Quantifiers – Questions

- ① **Computing:** Express “Every user has at least one valid login” using quantifiers.
- ② What is the negation of

$$\forall x P(x)?$$

- ③ Are $\forall x \exists y P(x, y)$ and $\exists y \forall x P(x, y)$ equivalent?
- ④ Express “There exists an integer whose square is 9.”
- ⑤ Negate

$$\exists x \forall y P(x, y).$$

3. Predicates and Quantifiers – Answers

1

$$\forall x (User(x) \rightarrow \exists y ValidLogin(y, x)).$$

2

$$\neg \forall x P(x) \equiv \exists x \neg P(x).$$

3

No. The first allows a different y for each x ; the second requires one common y for every x .

4

$$\exists x \in \mathbb{Z} (x^2 = 9).$$

5

$$\forall x \exists y \neg P(x, y).$$

4. Truth Tables – Questions

- ① **Computing:** A security rule is

$$A = (p \wedge q) \vee \neg r.$$

How many rows are required in its truth table?

- ② When is an implication $p \rightarrow q$ true?
- ③ Prove using a truth table:

$$\neg(p \wedge q) \equiv \neg p \vee \neg q.$$

- ④ Define tautology, contradiction, and contingency.
- ⑤ Is $p \rightarrow q$ equivalent to $q \rightarrow p$?

4. Truth Tables – Answers

- ① There are three variables. Therefore,

$$2^3 = 8$$

rows are required.

- ② It is false only when p is true and q is false.
- ③ Evaluate both sides for all four combinations of p, q . Their truth values are identical, so they are logically equivalent.
- ④ A tautology is always true. A contradiction is always false. A contingency is sometimes true and sometimes false.
- ⑤ No. $p \rightarrow q$ and $q \rightarrow p$ are generally different propositions.

5. First-Order Logic Satisfiability – Questions

- ① **Computing:** Is

$$\forall x (User(x) \rightarrow \exists y Login(y, x))$$

satisfiable?

- ② What does it mean for a formula to be satisfiable?

- ③ Is

$$\forall x P(x) \wedge \exists x \neg P(x)$$

satisfiable?

- ④ How can an existential formula be shown to be satisfiable?

- ⑤ Is every satisfiable formula valid?

5. First-Order Logic Satisfiability – Answers

- ① Yes. Construct an interpretation where every user has at least one login.
- ② A formula is satisfiable if there exists at least one interpretation in which it is true.
- ③ No. The first part says every object satisfies P , while the second requires one object that does not satisfy P .
- ④ Give a suitable domain and provide a witness satisfying the existential condition.
- ⑤ No. Satisfiable means true in at least one interpretation; valid means true in every interpretation.

6. Pattern of Proof – Questions

- ① **Computing:** A recursive algorithm changes n to $n - 1$. How can induction help prove its correctness?
- ② How do you prove a universal statement?
- ③ How do you prove an existential statement?
- ④ How can a universal statement be disproved?
- ⑤ How does the form of a mathematical statement suggest the proof technique?

6. Pattern of Proof – Answers

- 1 Prove correctness for the base case, then assume correctness for k and prove it for $k + 1$.
- 2 Choose an arbitrary object and prove the property for it.
- 3 Provide a specific witness and prove that it satisfies the required property.
- 4 Give one valid counterexample.
- 5 Universal statements usually suggest direct proof or induction; existential statements suggest finding a witness; implications suggest proving the consequent from the antecedent.

7. Proofs by Cases – Questions

- 1 **Computing:** A program behaves differently for even and odd inputs. How can its correctness be proved?
- 2 Prove that $n(n + 1)$ is always even.
- 3 Prove that $|x| \geq 0$ using cases.
- 4 How could a proof involving an integer modulo 3 be divided into cases?
- 5 What is the most important requirement when using proof by cases?

7. Proofs by Cases – Answers

- ❶ Consider the two exhaustive cases: n even and n odd. Prove correctness independently in each case.
- ❷ If n is even, the product is even. If n is odd, $n + 1$ is even. Hence the product is always even.
- ❸ If $x \geq 0$, then $|x| = x \geq 0$. If $x < 0$, then $|x| = -x > 0$.
- ❹ Consider:

$$n \equiv 0, 1, 2 \pmod{3}.$$

- ❺ The cases must be exhaustive and should not leave out any possible situation.

8. Proof of an Implication – Questions

- ① **Computing:** A cache key being valid implies that the corresponding record exists. How can this implication be proved?
- ② Prove: If an integer is divisible by 6, then it is divisible by 3.
- ③ What is the contrapositive of “if n^2 is odd, then n is odd”?
- ④ Is the converse of an implication automatically true?
- ⑤ What is vacuous truth?

8. Proof of an Implication – Answers

- 1 Assume the cache key is valid. Use the definition of validity to show that a corresponding stored record must exist.

- 2 Let $n = 6k$. Then

$$n = 3(2k),$$

so n is divisible by 3.

- 3 The contrapositive is:

$$n \text{ even} \rightarrow n^2 \text{ even.}$$

- 4 No. The converse requires a separate proof.
- 5 An implication is true whenever its antecedent is false, regardless of the consequent.

9. Proof by Contradiction – Questions

- 1 **Computing:** Why can a graph path that visits more vertices than possible without repetition be used to derive a contradiction?
- 2 What is the basic method of proof by contradiction?
- 3 Give the main idea for proving that $\sqrt{2}$ is irrational.
- 4 Why does the statement “there is no largest integer” lead to contradiction if a largest integer is assumed?
- 5 When is contradiction especially useful?

9. Proof by Contradiction – Answers

- ① A finite graph has only finitely many vertices. Assuming an excessively long simple path forces a repeated vertex, contradicting simplicity.
- ② Assume the negation of the desired conclusion and derive an impossible statement.
- ③ Assume $\sqrt{2} = a/b$ in lowest terms. Squaring gives $a^2 = 2b^2$, forcing both a and b to be even, a contradiction.
- ④ If n were largest, then $n + 1$ would be a larger integer.
- ⑤ It is useful when directly proving the statement is difficult but its negation naturally creates an impossibility.

10. Proving IFF – Questions

- ① **Computing:** A binary tree is balanced iff the height difference of its subtrees is at most 1. How would you prove this?
- ② What does $P \iff Q$ mean?
- ③ Prove:
$$n \text{ is even} \iff n^2 \text{ is even.}$$
- ④ How can set equality be proved using iff?
- ⑤ Why is proving only one direction insufficient?

10. Proving IFF – Answers

- 1 Prove both directions: $\text{balanced} \rightarrow \text{height difference at most 1}$, and $\text{height difference at most 1} \rightarrow \text{balanced}$.
- 2 It means both implications hold:

$$P \rightarrow Q \quad \text{and} \quad Q \rightarrow P.$$

- 3 If $n = 2k$, then $n^2 = 4k^2$ is even. Conversely, if n^2 is even, n must be even.
- 4 Show

$$x \in A \iff x \in B$$

for every element x .

- 5 One direction only establishes implication, not equivalence.

11. Sets – Questions

- 1 **Computing:** If A is all users and B is administrators, what does $A - B$ represent?
- 2 Define union and intersection.
- 3 What does $A \subseteq B$ mean?
- 4 How many subsets does a set containing n elements have?
- 5 State De Morgan's laws for sets.

11. Sets – Answers

1

$$A - B$$

represents users who are not administrators.

2

$$A \cup B$$

contains elements in either set, while

$$A \cap B$$

contains elements common to both.

3 Every element of A is also an element of B .

4

$$2^n.$$

5

$$(A \cup B)^c = A^c \cap B^c$$

and

$$(A \cap B)^c = A^c \cup B^c$$

12. Proving Set Equations – Questions

- ① **Computing:** Show that the database query “ $(A \text{ OR } B) \text{ AND NOT } C$ ” is equivalent to

$$(A \wedge \neg C) \vee (B \wedge \neg C).$$

- ② Prove:

$$A - B = A \cap B^c.$$

- ③ Prove:

$$A \cup (A \cap B) = A.$$

- ④ What is the two-inclusion method?

- ⑤ Prove:

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C).$$

12. Proving Set Equations – Answers

- ① By distributivity:

$$(A \vee B) \wedge \neg C = (A \wedge \neg C) \vee (B \wedge \neg C).$$

②

$$x \in A - B \iff x \in A \wedge x \notin B \iff x \in A \cap B^c.$$

- ③ By absorption:

$$A \cup (A \cap B) = A.$$

- ④ Prove both

$$A \subseteq B$$

and

$$B \subseteq A.$$

Then $A = B$.

- ⑤ Use distributivity:

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C).$$

13. Russell's Paradox – Questions

- 1 **Computing:** Why does the collection of all data structures that do not contain themselves create a Russell-type paradox?
- 2 State Russell's paradox.
- 3 Why does unrestricted set formation cause problems?
- 4 How does axiomatic set theory avoid the paradox?
- 5 Why does

$$R \in R \iff R \notin R$$

produce a contradiction?

13. Russell's Paradox – Answers

- ① Let R contain all structures that do not contain themselves. Asking whether R contains itself produces the contradiction.
- ② Consider

$$R = \{x : x \notin x\}.$$

Then $R \in R$ iff $R \notin R$.

- ③ It allows definitions such as “the set of all sets having property P ”, which can be self-referential.
- ④ Modern axiomatic set theory restricts how sets can be constructed.
- ⑤ If $R \in R$, the definition says $R \notin R$. If $R \notin R$, the definition says $R \in R$. Both possibilities contradict themselves.

14. Well-Ordering Principle – Questions

- 1 **Computing:** Why does repeatedly selecting a smaller positive integer eventually terminate?
- 2 State the Well-Ordering Principle.
- 3 How is the minimal-counterexample method related to well-ordering?
- 4 Are all integers well-ordered under the usual $\leq$ relation?
- 5 What is the relationship between well-ordering and induction?

14. Well-Ordering Principle – Answers

- ① The positive integers are well-ordered, so an infinite strictly decreasing sequence of positive integers cannot exist.
- ② Every nonempty subset of the positive integers has a least element.
- ③ If counterexamples exist, choose the smallest one. Then derive a contradiction by constructing a smaller counterexample.
- ④ No. The set of all integers has no least element.
- ⑤ The Well-Ordering Principle and the Principle of Mathematical Induction are equivalent over the positive integers.

15. Induction – Questions

- ① **Computing:** A loop processes one array element per iteration. How can induction prove that after k iterations exactly k elements have been processed?
- ② State the Principle of Mathematical Induction.
- ③ Prove:

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

- ④ What is the induction hypothesis?
- ⑤ What are common mistakes in induction proofs?

15. Induction – Answers

- 1 Prove the property for the first iteration, assume it after k iterations, then show it after $k + 1$ iterations.
- 2 Prove a base case and prove that

$$P(k) \rightarrow P(k + 1).$$

- 3 Base:

$$1 = \frac{1(2)}{2}.$$

Assume true for k . Then

$$\frac{k(k + 1)}{2} + (k + 1) = \frac{(k + 1)(k + 2)}{2}.$$

- 4 It is the assumption that $P(k)$ is true in the induction step.
- 5 Missing the base case, assuming the conclusion directly, or failing to prove the step from k to $k + 1$.

16. Invariants – Questions

- ① **Computing:** What loop invariant can be used for insertion sort?
- ② What is a loop invariant?
- ③ What are the three major stages of using an invariant?
- ④ Why are invariants important in algorithm correctness?
- ⑤ Does proving an invariant alone prove that an algorithm terminates?

16. Invariants – Answers

- ➊ Before each iteration, the processed portion of the array is sorted and contains exactly the same elements as the corresponding original portion.
- ➋ A property that remains true before and after every loop iteration.
- ➌ Initialization, maintenance, and termination.
- ➍ They provide a structured way to prove that an algorithm maintains correctness throughout execution.
- ➎ No. Termination must be established separately.

17. Strong Induction – Questions

- ① **Computing:** For

$$T(n) = T(n-1) + T(n-2),$$

why can strong induction be useful?

- ② State the Principle of Strong Induction.
- ③ Prove that every integer $n \geq 2$ can be expressed as a product of primes.
- ④ Why can strong induction require multiple base cases?
- ⑤ What is the difference between ordinary and strong induction?

17. Strong Induction – Answers

- ① The recurrence depends on multiple earlier values, so assuming all previous cases makes the induction step natural.
- ② Assume $P(1), P(2), \dots, P(k)$ are true and prove $P(k + 1)$.
- ③ If n is prime, it is already a product of one prime. If composite, write $n = ab$ where $a, b < n$ and apply the induction hypothesis to both.
- ④ The proof of $P(k + 1)$ may depend on several earlier values rather than only $P(k)$.
- ⑤ Ordinary induction assumes only $P(k)$; strong induction assumes all previous cases up to k .

18. Structural Induction – Questions

- 1 **Computing:** For a binary expression tree, prove that the number of leaves equals the number of internal binary nodes plus one.
- 2 What is structural induction?
- 3 What are the main cases when proving a property of recursively defined binary trees?
- 4 How is structural induction different from ordinary induction?
- 5 Why is structural induction suitable for recursive data structures?

18. Structural Induction – Answers

- ① Base case: a leaf has one leaf and zero internal nodes. For an internal node with two subtrees, combine the induction hypotheses:

$$(L_1 + L_2) = (l_1 + 1) + (l_2 + 1) = l_1 + l_2 + 2.$$

Since the new internal node adds one,

$$L = l + 1.$$

- ② Structural induction proves properties by following the recursive construction of an object.
- ③ Base case: leaf. Recursive case: combine the two subtrees.
- ④ Ordinary induction is based on integer progression; structural induction is based on the construction rules of recursive objects.
- ⑤ Recursive structures such as trees, expressions, and lists are naturally generated by construction rules.

Thank You

Mathematics of Computing

CUSAT M.Tech – AI & Software Engineering

Sayona C Sajan